



Universidad Autónoma de Nuevo León

Facultad de Ciencias Físico Matemáticas



PROGRAMACIÓN PARA CIBERSEGURIDAD

Producto Integrador de Aprendizaje

Integrantes

2135609	Castillo Cruz, Luis Emiliano
2145951	Murillo Soto, Alexa Valeria
2149255	Navarro Hernández, Ian Ubaldo
2146991	Pérez Álvarez, David Roberto
2135659	Santiesteban Moreira, Verónica Rubí

Grupo: 061

Docente: Osvaldo Habib González González

San Nicolas de los Garza, Nuevo León, México a 06 de mayo de 2026

Contenido

Introducción al problema	3
Descripción de la herramienta desarrollada	3
Arquitectura del sistema	4
Explicación de los scripts	4
Python: SystemAuditMainpy.py	4
PowerShell: SystemAuditMain.ps1	5
Bash: SystemAuditMainsh.py	6
Explicación de los módulos	6
Python: SystemAuditModulepy.py	6
PowerShell: SystemAuditModule.psm1	8
Bash: SystemAuditModulesh.py	8
Proceso de automatización	9
Manejo de errores y logging	10
Justificación de datos recolectados	10
Análisis del equipo	11
Conclusiones	12

Introducción al problema

Hoy en día, la seguridad en redes es muy importante, pero no siempre es fácil saber qué está pasando dentro de una red local. A veces no queda claro qué dispositivos están conectados, qué puertos están abiertos o si existe algún tipo de riesgo.

Nuestro proyecto busca solucionar eso con una herramienta sencilla que permite hacer un análisis básico de la red sin necesidad de usar programas complicados o difíciles de configurar. La idea es que cualquier usuario pueda identificar los equipos activos y revisar cuáles servicios están funcionando, además de tener una visión general del estado de la red sin tanta complicación.

Se tomó en cuenta que la herramienta fuera práctica y accesible, por lo que se pensó en que pudiera ejecutarse en diferentes sistemas operativos, permitiendo que más personas pudieran utilizarla sin importar el sistema operativo en el que trabajen.

Descripción de la herramienta desarrollada

Se trata de un sistema de auditoría de red que funciona desde la consola y permite realizar distintos análisis de forma sencilla y directa.

Entre sus funciones principales se encuentra la detección de dispositivos conectados a la red mediante ARP, el escaneo de puertos usando la técnica SYN para identificar cuáles están abiertos, y la obtención de información de servicios activos. También permite identificar el fabricante de un dispositivo a partir de su dirección MAC, lo cual puede ser útil para reconocer el tipo de equipo.

Otra característica importante es que calcula un nivel de riesgo para cada host analizado, lo que facilita detectar posibles vulnerabilidades o configuraciones inseguras dentro de la red.

Además, la herramienta permite guardar los resultados en diferentes formatos como CSV, JSON o HTML, lo que ayuda a organizar la información.

Arquitectura del sistema

Este sistema está organizado de forma modular, lo que facilita entender cómo funciona y también hace más sencillo su mantenimiento o mejora en el futuro.

Por un lado, cuenta con una parte de interacción con el usuario, que consiste en un menú en consola desde donde se pueden seleccionar las diferentes opciones de análisis. Por otro lado, está la parte lógica del sistema, donde se realizan todas las operaciones importantes, como el escaneo de red, el análisis de puertos y el procesamiento de la información obtenida. Esta es la parte central del funcionamiento de los scripts.

También incluye una sección encargada de generar reportes, donde se pueden organizar los resultados y se exportan en distintos formatos, permitiendo que la información sea fácil de guardar, revisar o compartir. El sistema fue desarrollado en tres lenguajes diferentes: Python, PowerShell y Bash. Esto permite que la herramienta pueda ejecutarse en distintos sistemas operativos como Windows y Linux, adaptándose a cada entorno y haciendo que sea más flexible y accesible para diferentes usuarios.

Explicación de los scripts

Python: `SystemAuditMain.py`

Este script funciona como la interfaz principal del sistema de auditoría de red. Su propósito es mostrar un menú interactivo en la consola para que el usuario seleccione la función relacionada con escaneo y análisis de red que desee utilizar.

Cuenta con cuatro funciones principales:

- `mostrar_menu()`: muestra en pantalla las opciones disponibles, incluyendo escaneo ARP, escaneo SYN, exportación de resultados y cálculo de riesgo.
- `scan_y_guardar(puertos=None, banner=False)`: ejecuta un escaneo de red utilizando las funciones del módulo principal, además de que permite activar la captura de banners y personalizar los puertos que se vayan a analizar.
- `pedir_puertos()`: solicita al usuario una lista de puertos separados por comas.
- `main()`: controla todo el flujo del programa mediante el menú. Dependiendo de la opción seleccionada por el usuario, ejecuta escaneos, exporta resultados o consulta información específica.
 - Escaneo ARP de dispositivos conectados
 - Escaneo SYN de puertos
 - Escaneo con puertos personalizados
 - Captura de banners de servicios
 - Exportación de resultados en CSV, JSON y HTML
 - Consulta de fabricante mediante dirección MAC
 - Cálculo de riesgo de hosts detectados

PowerShell: [SystemAuditMain.ps1](#)

Este script funciona como la interfaz principal de auditoría para sistemas Windows utilizando PowerShell.

El programa presenta un menú interactivo con diferentes categorías de análisis, entre las que se encuentran: información del sistema, procesos, redes y conexiones, usuarios y privilegios, manejo de logs, etc.

Este script importa el módulo principal SystemAuditModule.psm1 y ejecuta funciones específicas dependiendo de la opción que eligió el usuario.

Incluye manejo de errores mediante bloques try/catch, evitando que el programa termine de forma inesperada.

Bash: SystemAuditMainsh.py

Contiene una implementación independiente orientada a sistemas Linux y Unix, los archivos están escritos en Python, pero fueron diseñados para ejecutarse y automatizar tareas dentro de Bash.

El propósito de este script es coordinar el flujo principal del programa y utilizar las funciones definidas dentro del módulo.

Explicación de los módulos

Python: SystemAuditModulepy.py

Este módulo contiene toda la lógica de red y análisis que utiliza el script principal.

Se emplean las siguientes librerías:

- Scapy: crea y envía de paquetes de red. Permite realizar escaneos ARP y SYN directamente sobre la red.
- Socket: utilizado para establecer conexiones TCP y obtener banners de servicios activos.
- Time: empleado para controlar tiempos de espera y mediciones con respecto al escaneo.

- `Json`: permite exportar los resultados en formato json.
- `Csv`: genera archivos CSV con la información obtenida.
- `Concurrent.futures.ThreadPoolExecutor`: permite realizar los escaneos utilizando múltiples hilos, mejorando el rendimiento del análisis.

Funciones principales:

- `obtener_red_local()`: obtiene la red local del equipo.
- `parse_puertos(puertos)`: convierte diferentes formatos de entrada en una lista válida de puertos.
- `get_vendor(mac)`: identifica al fabricante de un dispositivo a partir de los primeros bytes de la dirección MAC.
- `guess_os(ttl)`: realiza una estimación del sistema operativo utilizando el valor TTL de los paquetes recibidos.
- `risk_score(ports)`: calcula el riesgo dependiendo de los puertos abiertos.
- `arp_scan(rango_red=None, timeout=3)`: realiza un escaneo ARP para detectar dispositivos activos dentro de la red local.
- `detectar_servicio(ip, puerto)`: intenta conectarse a un puerto abierto y capturar información del servicio activo.
- `scan_puertos_ip(host, puertos, timeout=1, banner=False)`: realiza un escaneo SYN sobre un host específico para detectar puertos abiertos.

Adicionalmente, cuenta con tres funciones de exportación en distintos formatos, lo que nos permite generar reportes legibles y fáciles de compartir.

PowerShell: [SystemAuditModule.psm1](#)

Este módulo contiene toda la lógica de recopilación y auditoría del sistema operativo.

Funciones principales:

- Write-AuditLog: registra eventos y errores dentro del archivo de log del sistema.
- Get-SystemCache: obtiene información del sistema utilizando Get-ComputerInfo y almacena los resultados en caché.
- Get-SystemDetails: recopila información detallada del equipo.
- Get-InstalledSoftware: obtiene la lista de software instalado leyendo el registro de Windows.
- Test-DiskUsage: analiza el porcentaje del disco duro y detecta estados críticos.

De forma general, recopila información relacionada con conexiones, usuarios locales, permisos, estado de seguridad, etc.

Bash: [SystemAuditModulesh.py](#)

Este módulo incluye funciones relacionadas con procesamiento de contenido, validación de datos, extracción de información, filtrado de resultados, automatización de tareas y generación de estadísticas.

Este módulo tiene las siguientes funcionalidades principales:

- Extracción y análisis de contenido
- Procesamiento de texto y estructuras HTML
- Filtrado de información inválida
- Seguimiento de elementos procesados
- Generación de métricas y estadísticas

- Manejo automatizado de resultados

Proceso de automatización

El proceso de automatización se implementa mediante una arquitectura modular distribuida en tres entornos: Python, Bash y PowerShell, donde cada uno cuenta con un archivo principal (Main) y un módulo de funciones (Module).

En todos los casos, el archivo Main actúa como punto de entrada del sistema, encargándose de coordinar la ejecución de las funciones definidas en su respectivo módulo. Este diseño permite separar la lógica de control de la lógica funcional, facilitando el mantenimiento y la reutilización del código.

El proceso automatizado consiste en ejecutar diferentes rutinas de auditoría que recopilan información del sistema, red, procesos, usuarios y seguridad.

Cada implementación sigue un flujo similar:

- Recolección de información del sistema
- Análisis de datos (procesos, red, seguridad)
- Evaluación de riesgos
- Generación de resultados estructurados
- Exportación en distintos formatos

Este enfoque permite que el sistema sea adaptable a distintos entornos, manteniendo una lógica común de auditoría independientemente del lenguaje utilizado.

Manejo de errores y logging

El sistema implementa manejo de errores en los tres entornos mediante los mecanismos propios de cada lenguaje.

En Python, se utilizan bloques try-except para controlar excepciones durante escaneos de red, conexiones y procesamiento de datos.

En PowerShell, se emplean bloques try-catch en cada función crítica, asegurando que errores no detengan la ejecución completa.

En Bash, se manejan errores mediante validaciones y control de ejecución de comandos.

En el caso de PowerShell, se implementa además un sistema de logging mediante la función Write-AuditLog, que registra eventos en un archivo (SystemAudit.log). Este registro incluye tanto ejecuciones exitosas como errores, lo que permite dar seguimiento al comportamiento del sistema.

Justificación de datos recolectados

Los datos recolectados en el sistema fueron seleccionados con el objetivo de proporcionar una visión integral del estado de un equipo desde una perspectiva de auditoría y seguridad.

Entre los principales datos obtenidos se incluyen:

- Información del sistema (CPU, RAM, sistema operativo)
- Procesos en ejecución y consumo de recursos
- Conexiones de red y puertos abiertos
- Usuarios y privilegios
- Eventos de seguridad
- Estado de firewall y antivirus

- Servicios críticos del sistema
- Persistencia (programas de inicio)
- Dispositivos en red (en Python mediante ARP scan)
- Servicios expuestos y posibles vulnerabilidades

Además, se implementan mecanismos de análisis como:

- Detección de conexiones sospechosas
- Estimación de sistema operativo
- Cálculo de riesgo (risk score) basado en condiciones detectadas

La recopilación de estos datos permite identificar posibles vulnerabilidades, configuraciones inseguras y comportamientos anómalos, facilitando la toma de decisiones para mejorar la seguridad del sistema.

Análisis del equipo

Durante el desarrollo se hicieron varias pruebas para verificar si la herramienta realmente funcionaba bien en una red y con esto se pudo comprobar que detecta correctamente los dispositivos conectados, los puertos abiertos y los servicios activos. También se revisó cómo funciona el cálculo del nivel de riesgo, y sirvió para identificar de forma rápida qué equipos podrían ser más vulnerables. En general, los resultados fueron claros y consistentes.

Además, se probaron las versiones en Python, PowerShell y Bash, notando que todas funcionan de forma similar, aunque cada una se adapta mejor a su sistema operativo.

Tiene área de mejoría y mejor optimización, pero cumple con sus tareas de forma correcta.

Conclusiones

En general, el proyecto logró su objetivo, ya que se desarrolló una herramienta sencilla pero útil para analizar redes.

Se pudo comprobar que con este tipo de herramientas es más fácil detectar posibles riesgos y entender mejor lo que pasa dentro de una red.

El script desarrollado permite realizar una auditoría básica pero integral de un equipo, combinando la recolección de información con un análisis orientado a la detección de riesgos. La implementación en múltiples lenguajes (Python, Bash y PowerShell) demuestra que la lógica de auditoría puede adaptarse a distintos entornos sin perder consistencia en los resultados.

Uno de los aspectos más relevantes es la integración de diferentes tipos de análisis, como el escaneo de red, la revisión de procesos, la identificación de configuraciones de seguridad y la detección de posibles anomalías. Además, el uso de métricas como el risk score y el estado de salud del sistema permite transformar datos técnicos en información más fácil de interpretar.

También se observa un enfoque hacia la automatización y eficiencia, mediante el uso de ejecución paralela en Python, reutilización de datos (caché) en PowerShell y estructuras modulares en todos los entornos. Sin embargo, el sistema aún puede ampliarse, por ejemplo, integrando bases de datos de vulnerabilidades reales (CVE) o mejorando el sistema de logging en todos los lenguajes.

En este proyecto se logró desarrollar un sistema de auditoría funcional y adaptable, capaz de recopilar, analizar y presentar información relevante sobre el estado de un equipo. A través del uso de diferentes tecnologías, se construyó una solución que no solo automatiza tareas, sino que también aporta una visión general de seguridad.

El resultado final demuestra la importancia de integrar múltiples fuentes de información para identificar riesgos y comprender el comportamiento del sistema.